Lab 02 Activity Sheet 02

Using MongoDB Compass/Atlas

Intro to CRUD Operations **Solution**

The objective of this tutorial is to understand how MongoDB CRUD operations work using MongoDB Compass/Atlas using the `restaurants` collection in the `ample_restaurants` database.

Lab Prerequisite:
Be familiar with the Week 2 lecture and resources prior to attending this activity.
Complete the Database Setup in Lab 02 Activity Sheet 01

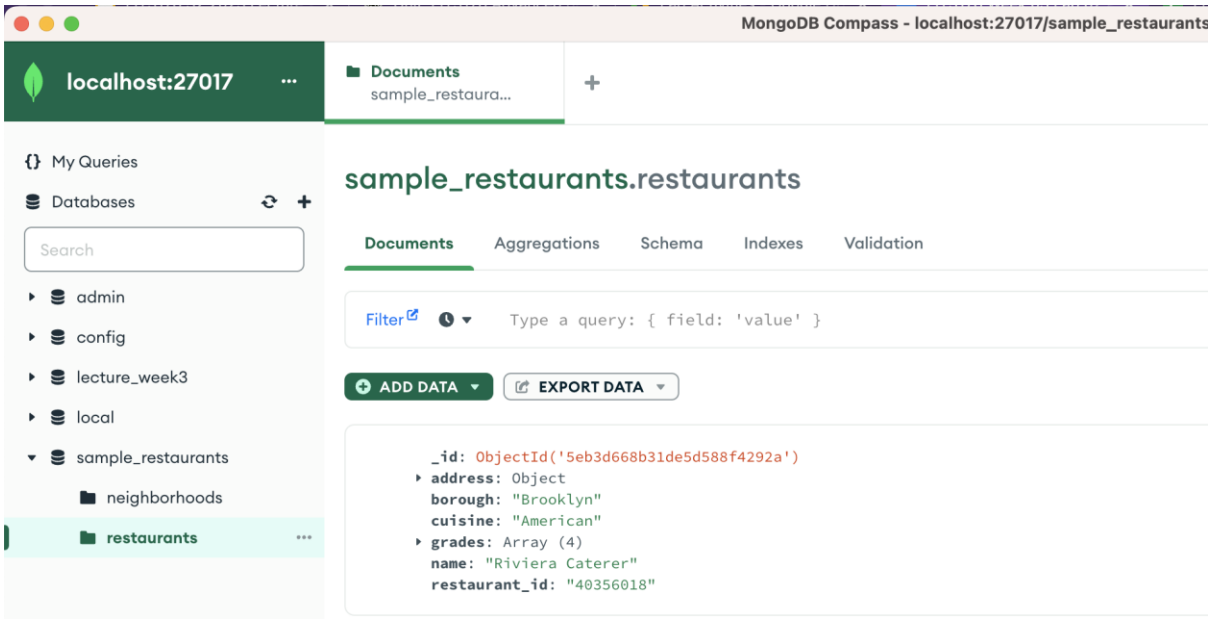
CRUD operations are:

- basic methods of interacting with a MongoDB server
- primary ways to manage the data in the database.

CRUD operations in MongoDB comprises of the following:

CREATE
READ
UPDATE
DELETE

Prior to running the CRUD operations you will need to navigate to the `restaurants` collection in the `sample_restaurants` database:



CREATE

is used to insert new documents in the database.
CREATE operations target a single collection
When creating documents if the specified collection doesn't exist, the create operation will create the collection when it's executed.

In the previous Lab 02 Activity Sheet 01you have already Inserted multiple Documents into the two Collections (`neighborhoods` and `restaurants`).

Tasks:

1. Insert the following document into the `restaurants` collection:

```
{ "address": {
"building": "1-B",
"coord": [ -73.98241999999999, 40.579505],
"street": "Stillwell Avenue",
"zipcode": "11224"
},
"borough": "Brooklyn",
"cuisine": "Indian",
"grades": [ { "date": { "$date": "2023-06-10T00:00:00.000Z" }, "grade": "A",
              "score": 7}],
"name": "Little India Caterer",
"restaurant_id": "140356018" }
```

To insert a single document into a collection using the GUI, you may use the following ways:

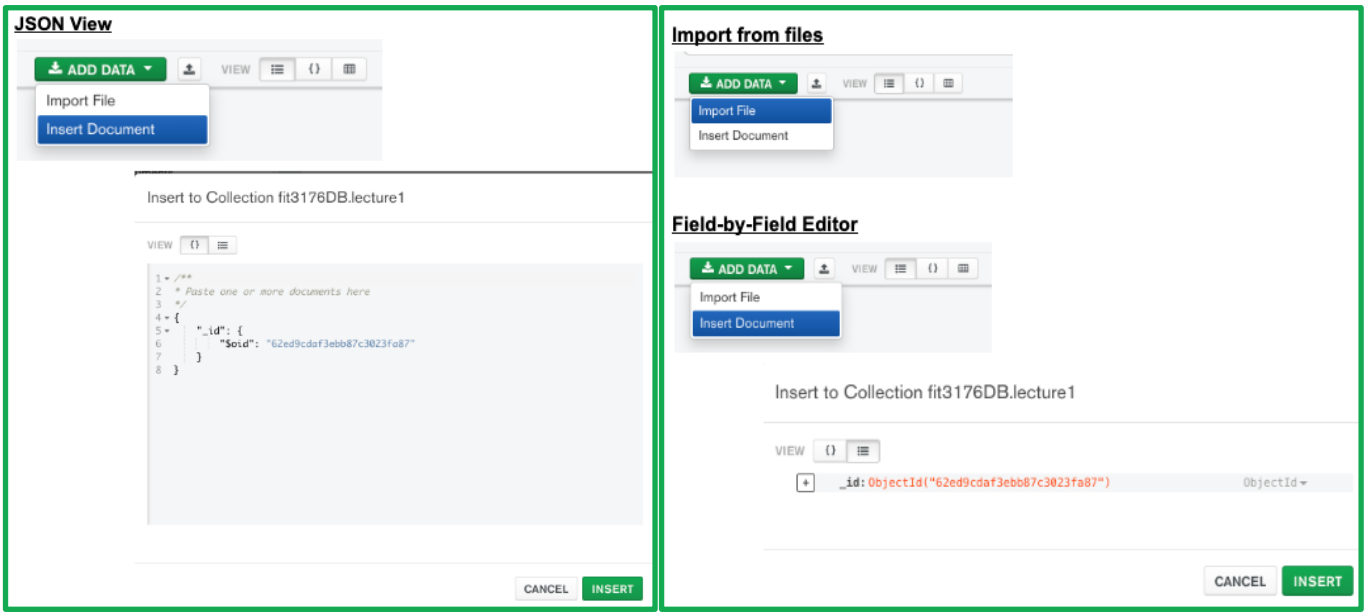
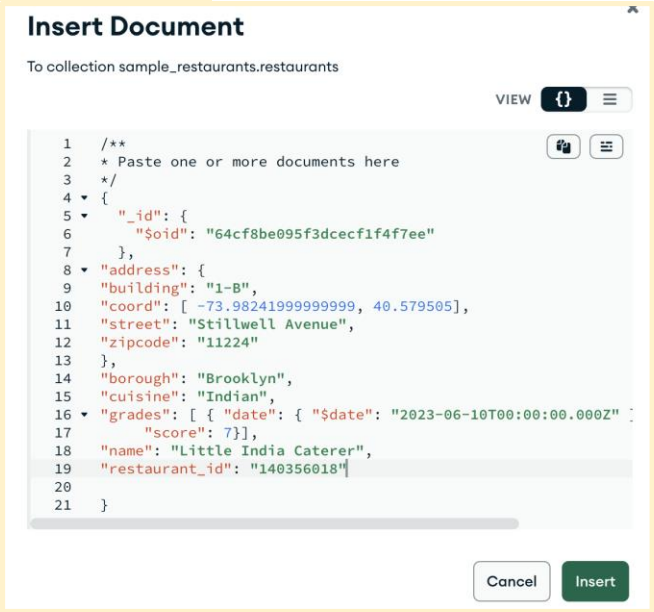


Figure: Inserting a single documen into a collection

For the _id allow MongoDB to set an automatic Object Id ($oid)

You can use the document clone feature to clone and modify `{"restaurant_id": "40356018"}`



READ

is used to query a document in the database.

READ operations uses query filters and criteria (find(), project()) to retrieve documents

Cursor Methods (e.g. limit(), skip(), sort(), collation(), maxTimeMs() etc.) can be used to change how many results are returned.

Figure: MongoDB Compass/Atlas GUI Query Bar

Query Bar Options	Purpose	Example (MongoDB Shell Code)
Find	finds documents that match a set of selection criteria	`db.restaurants.find({borough: 'Brooklyn'})`
Project	can specify which fields to return in the resulting data. By default, all fields are returned	`db.restaurants.find({borough: 'Brooklyn'}, {borough: 1, _id: 0})`

Sort	orders the documents in the result set	`db.restaurants.find({borough: 'Brooklyn'})` `.sort({borough:-1})`
Skip	controls the starting point of the results set.	`db.restaurants.find({borough: 'Brooklyn'}).skip(1)`
Limit	limits the number of documents in the result set.	`db.restaurants.find({borough: 'Brooklyn'}).limit(6)`
Collation	allows users to specify language-specific rules for string comparison, such as rules for letter case etc	`db.restaurants.find({borough: 'Brooklyn'})` `.collation({locale: "en_US"})`
MaxTimeMS	sets the cumulative time limit in milliseconds to process query bar operations. If the time limit is reached before the operation completes, Compass interrupts the operation.	`db.restaurants.find({borough: 'Brooklyn'}).maxTimeMS(5)`

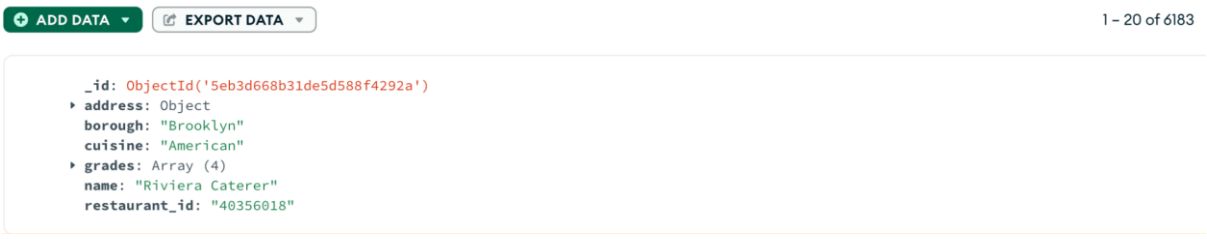
Table: MongoDB Compass/Atlas GUI Query Bar Options

Tasks:

Given that each document in the `restaurants` collection represents a restaurant, use the `restaurants` collection to retrieve the results of the following queries:

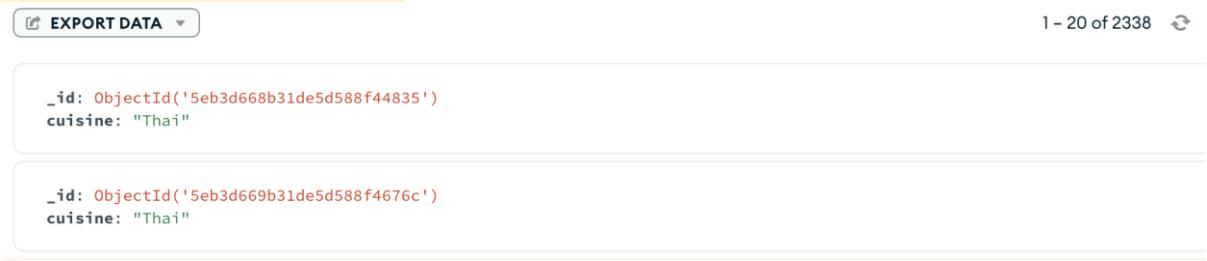
1. Display all restaurants serving `American` cuisine.

```
FILTER: { "cuisine":"American" }
```



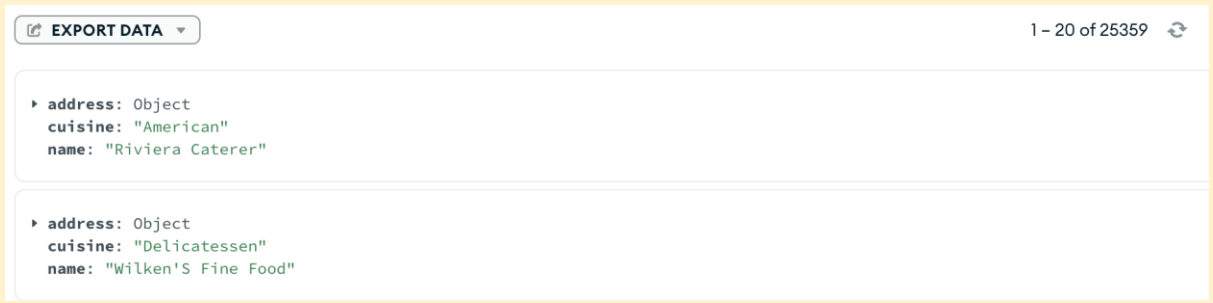
2. For all restaurants in the `Brooklyn borough` display only the `cuisine` field sorted in reverse alphabetical order

```
FILTER: { "borough":"Brooklyn" }
PROJECT: { "cuisine":1 }
SORT: {"cuisine": -1}
```



3. Display the `name`, `address` and `cuisine` for all restaurants excluding the `_id` from the results.

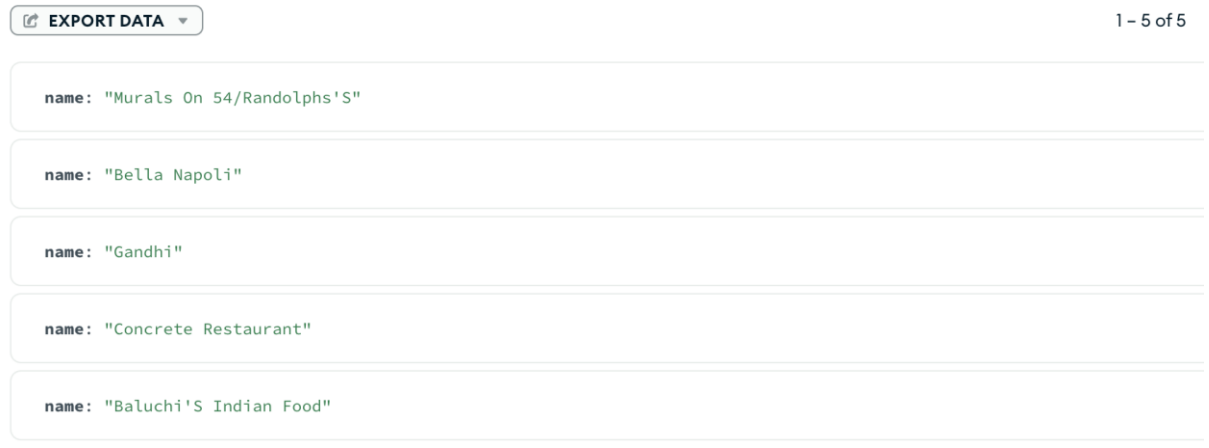
```
PROJECT: { "name":1, "address":1, "cuisine":1, "_id":0 }
```



Using the Query and Projection Operators answer the following:

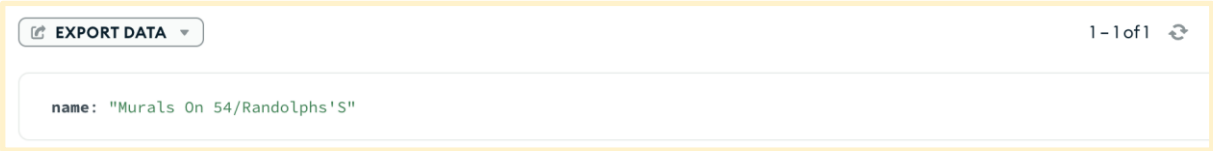
4. Display the names of all restaurants who had a `grades score` of greater than or equal to 90 excluding the `_id` from the results.

```
FILTER: {"grades.score": { $gte: 90 } }
PROJECT: { "name":1, "_id":0 }
```



5. Among the restaurants who had at least one `grades score` of less than 90 (note that `grades` is an array and can have other scores greater than 90) , display the name of the restaurant which had the highest score (excluding the `_id`).

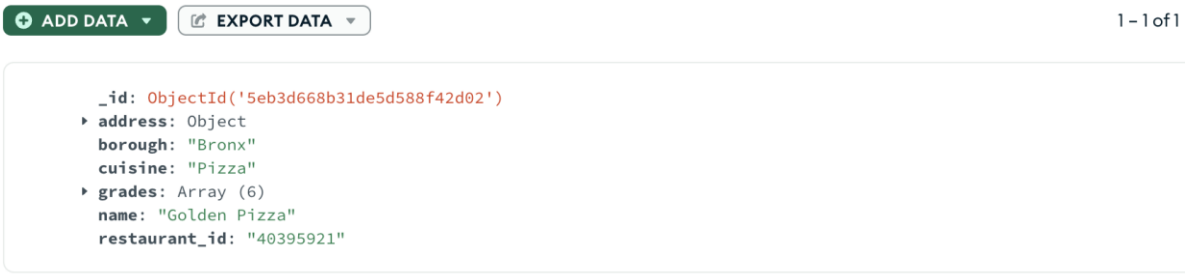
```
FILTER: { "grades.score": { $lt: 90 } }
PROJECT: {"name": 1, "_id": 0}
SORT: { "grades.score": -1 }
LIMIT: 1
```



6. Display all fields of the 4th highest `grades score` restaurant who had a `grades score` equal to 50 and did not serve `American` and `Indian` `cuisine`.

```
FILTER: {'grades.score': {$eq: 50}, "cuisine": {$nin:[
"American", "Indian" ]}}
SORT: { "grades.score": -1 }
```

```
SKIP: 3
LIMIT: 1
```



UPDATE

is used to modify existing documents in the database.

Similar to CREATE, UPDATE operations operate on a single collection, and they are atomic at a single document level.

An update operation takes filters and criteria to select the documents you want to update.

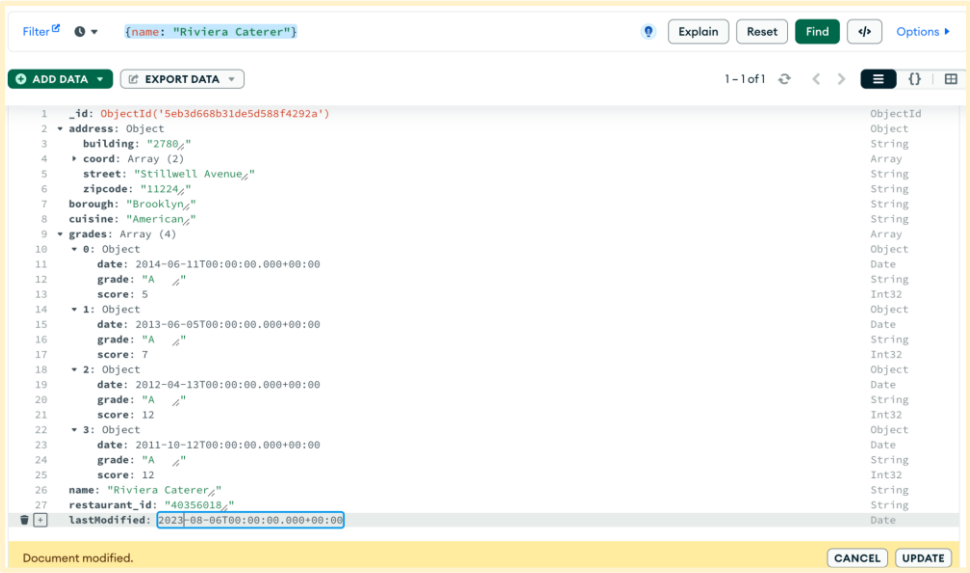
To track updates the Last Modified Date can be added

Note: when updating documents, updates are permanent and can't be rolled back. This applies to delete operations as well.

Task:

Update the `grades date` **for the restaurant with** `restaurant_id 40356018` **and** `score 5` **to** `2014-06-11.`

Note: add a field called `lastModified` (with the value as today's date) in each to keep track of the update Use the date data type and insert the value with today's date.



DELETE

used to remove documents in the database.

DELETE operations operate on a single collection, like update and create operations.

DELETE operations are also atomic for a single document.

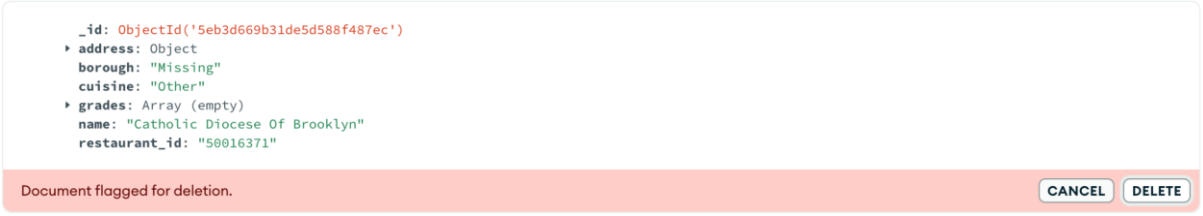
Filters and criteria can be used in DELETEs to specify which documents to delete from a collection.

The filter options rely on the same syntax that read operations utilize.

Task:

Find and remove the document(s) with `Missing borough`, `Other cuisine` **and no** `grades`

```
FILTER: { "borough":"Missing", "cuisine": "Other", "grades": {$size: 0}}
```



Challenge Questions:

Q1. Use the MongoDB documentation to list the components of the 12-byte ObjectId().

A. The 12-byte ObjectId consists of:

- A 4-byte timestamp, representing the ObjectId's creation, measured in seconds since the Unix epoch.
- A 5-byte random value generated once per process. This random value is unique to the machine and process.
- A 3-byte incrementing counter, initialized to a random value.

THE END